

CI/CD, Drift Detection & Provider Authentication

WEEK 4

Learning Objectives

- ▶ Understand Terraform CI/CD workflows
- ▶ Differentiate local vs CI execution
- ▶ Detect and reconcile infrastructure drift
- ▶ Configure secure provider authentication
- ▶ Implement GitHub Actions with AWS OIDC
- ▶ Build production-ready Terraform pipelines

CI/CD WORKFLOWS

Introduction to Terraform CI/CD

What is CI/CD in Terraform?

- ▶ CI/CD in Terraform refers to the practice of automating the lifecycle of infrastructure management using Infrastructure as Code (IaC) in CI/CD pipelines.
- ▶ Continuous Integration(CI): CI focuses on validating code quality before it is merged into the main branch.
- ▶ Continuous Delivery/Deployment (CD): CD focuses on safely applying the changes to your actual environment.

Popular Tools

- ▶ GitHub Actions, GitLab CI/CD, Jenkins, and Azure DevOps.

Benefits

- ▶ Consistency
- ▶ Automation
- ▶ Peer review
- ▶ Auditability
- ▶ Reduced manual errors

Local Workflow vs CI/CD Workflow

Feature	Local Workflow	CI/CD Workflow
Trigger	Manual (Human typing)	Automatic (Git event)
State Storage	Local Disk (risky)	Remote Backend (safe)
Secrets	Local Env Vars	Secret Manager / OIDC
Review Process	Over the shoulder / Screen share	Code review on PR
History	Hard to track who did what	Full audit log in CI system
Approval	Interactive prompt	Automated or via "Manual Gates"

Terraform CI/CD Pipeline Flow

- ▶ A DevOps Engineer interacts with the Git repository
 - ▶ Pull Request (PR)
 - ▶ Push code to repo
 - ▶ Merge to Main/Production
- ▶ CI runs:
 - ▶ fmt , validate
 - ▶ Linting/Security Scanning
 - ▶ Security Scan
- ▶ Plan output reviewed
 - ▶ Peer Review:
- ▶ Approval Gate
 - ▶ Manual Approval and merged
- ▶ CD runs:
 - ▶ Apply workflow triggered
 - ▶ Infrastructure updated

GitHub Actions for Terraform CI/CD

Using GitHub Actions

- ▶ Native CI/CD integration with GitHub
- ▶ Event-driven workflows
- ▶ YAML-based pipeline definitions

Important Components

- ▶ hashicorp/setup-terraform
- ▶ Workflow triggers
- ▶ Terraform caching
- ▶ Environment protection rules

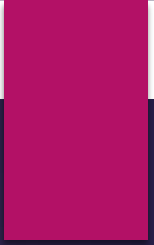
Managing Secrets in CI/CD

Methods for handling Secrets Injection in CI

- ▶ GitHub Secrets
- ▶ **External Secret Managers:**
 - ▶ HashiCorp Vault,
 - ▶ AWS Secrets Manager,
 - ▶ Azure Key Vault,
 - ▶ Google Secret Manager.
- ▶ Short-Lived Credentials OIDC federation (recommend)

Problems with Plain Secrets

- ▶ Credential leakage
- ▶ Rotation burden
- ▶ Long-lived credentials



DRIFT DETECTION & RECONCILIATION

What is Infrastructure Drift?

Understanding Drift

- ▶ Infrastructure drift (or configuration drift) occurs when the actual state of your real-world infrastructure no longer matches the desired state defined in your Infrastructure as Code (IaC) configuration.
- ▶ Causes of Drift:
 - ▶ Manual console changes (Manual Hotfixes): Direct infra edits through the cloud provider console (ClickOps)
 - ▶ External automation : Automated patches, OS updates, or software installations altering default settings.
 - ▶ Failed applies leave partial changes:

Result

- ▶ Terraform state $\neq$ Actual infrastructure

Intentional vs Unintentional Drift

Intentional Drift

- ▶ Emergency hotfixes
- ▶ Temporary operational changes

Unintentional Drift

- ▶ Unauthorized changes
- ▶ Configuration inconsistencies
- ▶ Manual console modifications

Detecting Drift with Terraform

Using Terraform Plan

- ▶ Terraform Detects
- ▶ Missing resources
- ▶ Configuration mismatches
- ▶ Manual modifications

Understanding Diff Output

- ▶ + create
- ▶ - destroy
- ▶ ~ modify

Automated Drift Detection

Scheduled CI Drift Checks

- ▶ Cron-triggered workflows
- ▶ Automated plan runs
- ▶ Slack or email notifications
- ▶ **Workflow Example**
- ▶ schedule:
 - cron: "0 */6 * * *"

Automation & Best Practices for Drift Detection

- ✓ **Scheduled CI Jobs:** Set up a CI job to run a terraform plan daily or hourly.
- ✓ **GitOps Approach:** Ensure the main branch (Source of truth) always reflects production.
- ✓ **Restrict Access:** Limit "Write" permissions in the cloud console.
- ✓ **ignore_changes Lifecycle:** For attributes that change naturally (like auto-scaling counts), use the lifecycle block to prevent false positive drift alerts

Terraform Drift Reconciliation Strategies 1

Revert to Configuration (Restore Source of Truth)

- ▶ Code-to-Cloud Approach (Enforce the Code)
- ▶ The code serves as the single source of truth

Process

- ▶ Run terraform plan
- ▶ Review differences
- ▶ Apply existing configuration

Use Case

- ▶ Use this when a manual change was unauthorized or accidental.

Terraform Drift Reconciliation Strategies 2

- ▶ **Update Configuration (Accept the Change)**

- ▶ Your code is updated to reflect the current reality, and future runs won't attempt to "fix" it
- ▶ Update Terraform configuration to match reality.

- ▶ **Process**

- ▶ Inspect actual infrastructure
- ▶ Modify HCL
- ▶ Run plan
- ▶ Apply changes

- ▶ **Use Case**

- ▶ Use this when a manual change (e.g., an emergency hotfix) was correct and should be permanent.

Terraform Drift Reconciliation Strategies 3

Advanced Automation (GitOps)

- ▶ For high-scale environments, manual reconciliation is often replaced by automated "self-healing" workflows.
- ▶ **Continuous Monitoring:** HCP Terraform (Cloud) or Spacelift automatically scan for drift every few hours and alert you if the state deviates.
- ▶ GitOps Reconciliation:
- ▶ **Driftctl:** open-source tool designed to scan cloud accounts

PROVIDER AUTHENTICATION

Terraform Provider Authentication

Why Authentication Matters

- ▶ Terraform providers require secure access to APIs.

Goals

- ▶ Least privilege
- ▶ Secure automation
- ▶ Short-lived credentials
- ▶ Auditability

Static Keys & Profiles

Traditional Authentication Methods

Environment Variables

AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY

Shared Credentials File

▶ ~/.aws/credentials

Suitable For

- ▶ Local development
- ▶ Temporary testing

Risks of Static Credentials

Security Challenges

- ▶ Long-lived secrets
- ▶ Difficult rotation
- ▶ Secret sprawl
- ▶ Large blast radius

CI/CD Risks

- ▶ Exposed repository secrets
- ▶ Insider threats
- ▶ Compliance challenges

Introduction to OIDC

What is OIDC?

- ▶ OpenID Connect federation enables:
- ▶ Temporary credentials
- ▶ Secretless authentication
- ▶ Federated identity access

Key Benefit

- ▶ No long-lived AWS keys stored in GitHub

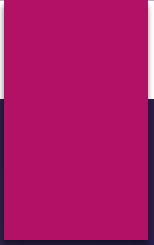
How OIDC Works

Authentication Flow

- ▶ GitHub Actions requests JWT
- ▶ AWS IAM validates token
- ▶ AWS issues temporary credentials
- ▶ Terraform uses assumed role

Trust Relationship

- ▶ GitHub ↔ AWS IAM



Configuring AWS OIDC Provider & GitHub Actions OIDC Authentication

Lab 4A — GitHub Actions CI Pipeline with OIDC